



Universidad de Oviedo
Universidá d'Uviéu
University of Oviedo

Design and Implementation of a Population Growth Simulator

Mario Orviz Viesca

Degree: Software Engineering

Tutor:

Celia Melendi Lavandera

Mario Quevedo de Anta

March 2026

Contents

1. Introduction	4
1.1. Motivation	4
2. Project Background	4
2.1. Domain Analysis and Current Solutions	4
2.2. Justification for Greenfield Development	4
2.3. Proposed Solution	4
3. Project Scope	4
3.1. Project Statement	4
3.2. System Boundaries	5
3.3. Out of Scope	5
4. Technology Stack	5
4.1. Programming Language — Python 3.13	5
4.2. Backend Framework — FastAPI	5
4.3. Data Validation — Pydantic v2	6
4.4. Numerical Computing — NumPy	6
4.5. Database — PostgreSQL 16	6
4.6. ORM — SQLAlchemy	6
4.7. Database Migrations — Alembic	6
4.8. Frontend Framework — Python Shiny	6
4.9. Containerisation — Docker and Docker Compose	7
4.10. Authentication — JWT	7
4.11. Testing — pytest	7
4.12. CI/CD — GitHub Actions	7
5. System Architecture	7
5.1. Overview	7
5.2. API Layer Internal Structure	8
5.3. Data Layer	9
5.4. Container Architecture	9
5.5. Design Decisions and Trade-offs	9
6. System Views	10
6.1. Context View	10
6.2. Building Block View	10
6.3. Runtime Views	11
7. API Design	19
7.1. Design Principles	19
7.2. Endpoint Reference	20
7.3. Authentication	20
7.4. Input Validation — Schemas	20
7.5. Output Serialisation — Records	21
7.6. Ephemeral vs. Stored Simulations	21
7.7. Ownership and Immutability Rules	21
7.8. Error Responses	21
8. Database Design	22
8.1. Schema Overview	22
8.2. Entity Details	23

8.3.	Migrations	24
8.4.	COMPADRE Seeding	24
9.	Simulation Engine	25
9.1.	Mathematical Background	25
9.2.	Deterministic Simulation	25
9.3.	Stochastic Simulation	25
9.4.	Validation	26
9.5.	Ephemeral vs. Stored Runs	26
9.6.	Key Properties	26
10.	Testing Strategy	27
10.1.	Philosophy	27
10.2.	Test Layout	27
10.3.	Unit Tests	27
10.4.	Integration Tests	28
10.5.	Running the Tests	29
10.6.	Coverage	29
11.	DevOps and CI/CD	29
11.1.	Overview	29
11.2.	CI Workflow (ci.yml)	29
11.3.	Security Workflow (security.yml)	30
11.4.	Docker Setup	31
11.5.	Local Development	31
12.	Cross-Cutting Concepts	32
12.1.	Security	32
12.2.	Error Handling	33
12.3.	Observability	33
12.4.	Reproducibility	34
12.5.	Data Integrity	34
12.6.	Testability	35
12.7.	Configuration Management	35
	Bibliography	35

1. Introduction

This is the beginning of my TFG at UniOvi.

1.1. Motivation

As a software engineering student, I want to solve...

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Quia ipsum suspendisse ultrices gravida. Risus commodo viverra maecenas accumsan. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

2. Project Background

2.1. Domain Analysis and Current Solutions

The project is situated within the field of biology, specifically focusing on population dynamics. Population growth is a fundamental concept in ecology and has significant implications for conservation, resource management, and understanding ecosystem health.

While legacy solutions for this problem exist, they often suffer from limitations such as technical debt and they're not designed to be easily extendable. These solutions may not be able to adapt to new requirements or incorporate additional features without significant rework, which can hinder their long-term viability and usefulness.

2.2. Justification for Greenfield Development

This project is being developed as a greenfield project, meaning that it is being built from scratch without any constraints imposed by existing code or systems. This approach allows for greater flexibility and creativity in the design and implementation of the solution.

By starting with a clean slate, the project can be tailored to meet specific requirements and can incorporate modern technologies and best practices without being hindered by legacy code. This can lead to a more efficient development process and a more robust and maintainable solution in the long run.

2.3. Proposed Solution

3. Project Scope

3.1. Project Statement

The project aims to develop a population growth simulator that models the dynamics of population growth under various conditions, including sources of stochasticity. The simulator will allow users to input parameters such as initial population size, growth rate, carrying capacity, and time steps to visualize how populations evolve over time. The goal is to create a tool that can be used for educational purposes, research, and to gain insights into population dynamics.

3.2. System Boundaries

The proposed system is a **standalone** application. It does not require synchronization or integration with external legacy databases or third-party APIs for its core operation. All data persistence and business logic are managed internally.

3.3. Out of Scope

The project will not include features such as:

- Real-time data synchronization with external sources.
- Integration with third-party APIs for data retrieval or analysis.
- Advanced machine learning algorithms for predictive modeling of population growth.
- User authentication and multi-user support.

4. Technology Stack

This chapter describes every technology selected for the project, the role each one plays, and the rationale behind choosing it over available alternatives.

4.1. Programming Language — Python 3.13

Python was chosen as the sole language for both the backend and the frontend. The decision rests on three pillars.

First, the scientific ecosystem: NumPy, the de-facto standard for numerical computing in Python, provides efficient n-dimensional array operations that map directly onto the matrix algebra at the core of the simulation engine. No equivalent library exists in other languages at the same level of maturity and community support.

Second, productivity: Python's concise syntax and rich standard library accelerate development. For an academic project with a fixed deadline this factor carries significant weight.

Third, ecosystem coherence: using a single language for the entire stack eliminates the friction of cross-language tooling, dependency management, and cognitive context switching.

Python 3.13 specifically was selected to benefit from the latest performance improvements to the interpreter and to match the version available in the CI runner.

4.2. Backend Framework — FastAPI

FastAPI [1] is the web framework that exposes the REST API. It was preferred over Flask and Django REST Framework for the following reasons.

- **Automatic OpenAPI documentation.** FastAPI generates a Swagger UI at `/docs` with zero additional code. This is valuable during development and for demonstrating the API to non-technical stakeholders.
- **Native Pydantic integration.** Request and response models are declared as Pydantic classes; FastAPI handles validation, serialisation, and error formatting automatically.
- **Async-first design.** FastAPI is built on top of Starlette and supports `async/await` natively, leaving the door open for future non-blocking I/O without a rewrite.
- **Type safety.** The heavy use of Python type annotations propagates throughout the codebase, catching errors at development time rather than at runtime.

4.3. Data Validation — Pydantic v2

Pydantic [2] is used exclusively for input and output modelling. All data arriving from HTTP requests is parsed and validated by *schemas* (input DTOs), while all data leaving the service layer is serialised through *records* (output models). This strict separation is detailed in Section 7.

Pydantic v2 was chosen over v1 because its Rust-backed validation core delivers substantially faster parsing, which matters when many matrices or simulation payloads are processed in a single request.

4.4. Numerical Computing — NumPy

NumPy [3] underpins the simulation engine. Population projection matrices are converted to `ndarray` objects and each time step is computed as a single matrix-vector product ($A @ v$), making use of NumPy’s optimised BLAS routines. The stochastic engine additionally relies on `numpy.random.default_rng` for reproducible pseudo-random matrix selection.

4.5. Database — PostgreSQL 16

PostgreSQL was selected as the relational database for the following reasons.

- **JSONB support.** Several columns (`matrix_a`, `matrix_u`, `matrix_f`, `result_history`, `stage_names`, `matrix_ids`) store variable-length nested structures. PostgreSQL’s native JSONB type indexes and queries these efficiently without sacrificing the guarantees of a relational engine.
- **Reliability and maturity.** PostgreSQL is a production-grade RDBMS with decades of development behind it, making it a safe long-term choice.
- **SQLAlchemy compatibility.** The ORM layer integrates with PostgreSQL through the well-maintained `psycopg2` driver.

Version 16 specifically introduces performance improvements to parallel query execution and JSONB handling.

4.6. ORM — SQLAlchemy

SQLAlchemy [4] acts as the bridge between Python objects and the database. The *declarative* mapping style (using `Base` and class-level column definitions) was chosen because it keeps model definitions readable and co-located with the Python type system.

The repository pattern is layered on top of SQLAlchemy sessions so that no business logic leaks into raw query code and no SQL leaks into the service layer.

4.7. Database Migrations — Alembic

Alembic [5] manages schema evolution. Every change to `db/models.py` is captured as a versioned migration script in `alembic/versions/`. Migrations are applied automatically by the container entrypoint on startup, ensuring that the database schema is always in sync with the running code without manual intervention.

4.8. Frontend Framework — Python Shiny

The frontend is a single-file Python Shiny [6] application. Shiny was chosen because it allows building reactive data applications entirely in Python, eliminating the need to introduce

JavaScript or a separate frontend language. This keeps the repository homogeneous and lowers the barrier to extending the UI.

The application never touches the database directly; all data flows through the REST API. This means the frontend is fully decoupled from the backend and can be replaced with any other HTTP client in the future.

4.9. Containerisation — Docker and Docker Compose

Docker [7] packages each service (API, frontend, database) into an isolated container with a deterministic environment. Docker Compose [8] orchestrates the three containers, defines the shared network, injects environment variables, and manages startup order with health checks. This setup means the entire stack can be started on any machine with a single command (`docker compose up -d --build`) and the behaviour is identical in development, CI, and production.

4.10. Authentication — JWT

Authentication is implemented with JSON Web Tokens (JWT) [9] signed with a server-side secret. Tokens carry the user identity and expire after seven days. The choice of JWT over server-side sessions avoids storing session state in the database and makes horizontal scaling straightforward.

4.11. Testing — pytest

pytest [10] was chosen as the test runner for its concise fixture system, rich assertion introspection, and the ability to parametrize tests. The suite is divided into unit tests (no database required, using `unittest.mock`) and integration tests (requiring a live PostgreSQL instance), allowing fast feedback during development without sacrificing end-to-end coverage.

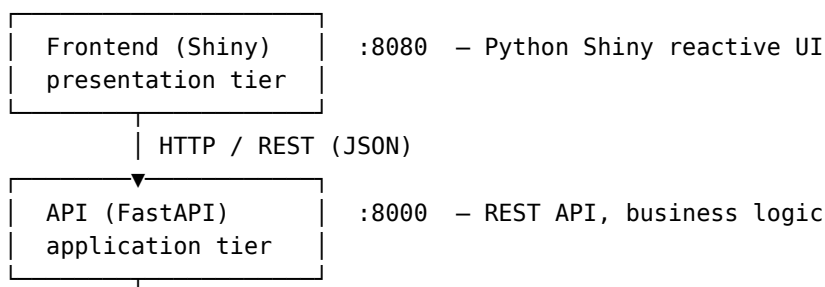
4.12. CI/CD — GitHub Actions

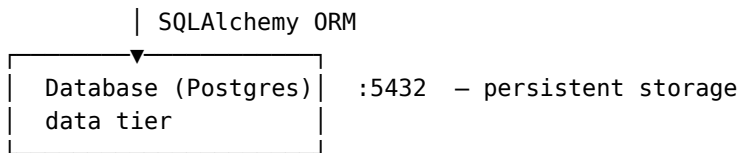
GitHub Actions [11] orchestrates the automated pipelines. Two workflow files are maintained: one for correctness (tests and coverage) and one for security (static analysis, dependency auditing, and container scanning). The rationale for this split is described in Section 11.

5. System Architecture

5.1. Overview

The system follows a classic three-tier architecture composed of a presentation layer, an application layer, and a data layer. Each tier runs as an independent Docker container and communicates only through well-defined interfaces.





The frontend never accesses the database directly. The API never accepts raw SQL from the outside. This strict separation means each tier can be scaled, replaced, or tested in isolation.

5.2. API Layer Internal Structure

Within the API tier a second, finer-grained layering is enforced: *controller* → *service* → *repository*. No layer may skip another.

5.2.1. Controllers (`api/controllers/`)

Controllers are responsible exclusively for HTTP concerns: parsing request bodies into schema objects, invoking the appropriate service method, and returning the resulting record with the correct HTTP status code. Controllers contain no business logic and perform no database access.

5.2.2. Services (`api/services/`)

Services contain all business logic: ownership checks, immutability rules for COMPADRE matrices, dimension validation across matrices, and the simulation algorithm itself. Services receive and return domain objects (schemas in, records out). They delegate all persistence to repositories.

5.2.3. Repositories (`api/repositories/`)

Repositories are the only layer permitted to interact with SQLAlchemy sessions. They expose a narrow, entity-centric interface (`get_by_id`, `create`, `update`, `delete`, `list`) and return raw ORM objects to the service layer. No business rule is encoded here.

5.2.4. Schemas and Records (`api/schemas.py`, `api/records.py`)

Two distinct families of Pydantic models cross the API boundary.

- **Schemas** (input DTOs) validate data arriving over HTTP. All input validation — field constraints, cross-field invariants — lives exclusively in schemas.
- **Records** (output models) represent business entities as they flow out of the service layer and into HTTP responses. They are constructed from ORM objects via `model_validate` with `from_attributes=True`.

This separation prevents accidental exposure of internal fields and makes the contract of each layer explicit.

5.2.5. Dependency Injection (`api/deps.py`)

FastAPI's dependency injection system is used to provide:

- A SQLAlchemy Session scoped to each request.
- Pre-built service instances that receive their repository dependencies.
- The `get_current_user` guard that validates JWT tokens and resolves the authenticated user.

The dependency tree is wired entirely in `deps.py`, keeping controllers and services free of any framework-specific boilerplate.

5.3. Data Layer

The database layer (db/) contains three components.

- **ORM models** (db/models.py) — SQLAlchemy declarative classes for User, PopulationMatrix, and SimulationRun.
- **Session factory** (db/session.py) — a SessionLocal factory consumed by the dependency injection layer.
- **COMPADRE seeder** (db/seed_compadre.py) — a startup script that populates the database with real-world population matrices from the COMPADRE Plant Matrix Database. The seeder is idempotent; re-running it on an already-seeded database is a no-op.

5.4. Container Architecture

Three containers are defined in `docker-compose.yml`.

Service	Image	Exposed port
db	postgres:16-alpine	5435 (host) → 5432 (container)
api	built from Dockerfile	8000
frontend	built from frontend/Dockerfile	8080

The api and frontend containers wait for db to pass its health check before starting. The api container runs `entrypoint.sh` on startup, which applies pending Alembic migrations and then seeds COMPADRE data before launching Uvicorn.

5.5. Design Decisions and Trade-offs

5.5.1. Single-language Stack

Running Python on all three tiers (API, frontend, seed scripts) eliminates build tool heterogeneity and makes it straightforward to share utility code. The trade-off is that Python is not the most performant language for CPU-bound work; however, matrix projection steps are delegated to NumPy's native BLAS routines, so this is not a practical bottleneck for the expected workloads.

5.5.2. Server-side Simulation

The decision to run the simulation algorithm on the server rather than in the browser was deliberate. Executing server-side guarantees that results are reproducible (the server controls the RNG), stored persistently, and identical regardless of the client device. It also prevents matrix data from being unnecessarily transmitted to the client before computation.

5.5.3. COMPADRE Read-only Constraint

Matrices seeded from the COMPADRE database are marked `source_type = "compadre"` and are rejected by the service layer if an update is attempted, regardless of the requesting user's identity. This protects the integrity of the reference dataset while still allowing users to create their own matrices with full edit rights.

6. System Views

The following views describe the system using the arc42 documentation framework. Diagrams are authored in PlantUML (docs/diagrams/) and rendered to SVG by running `bash docs/diagrams/render.sh` (requires PlantUML on PATH).

6.1. Context View

The context view shows the system as a black box and identifies the external actors that interact with it.

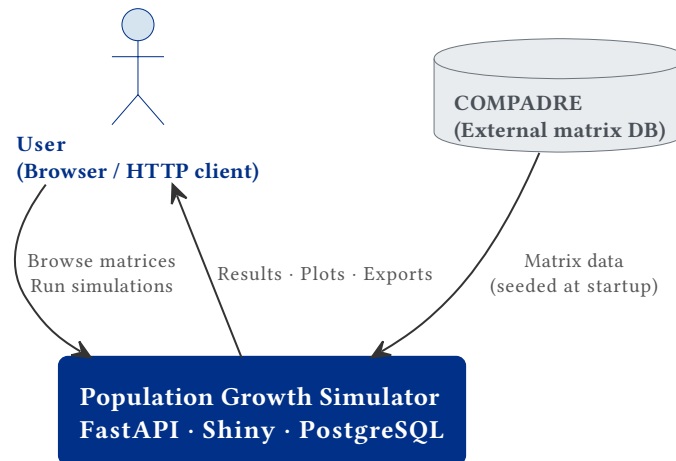


Figure 1: Context view — the system and its external actors

The system has two external actors.

- **User** — a scientist or student interacting with the application through a web browser. Users can browse matrices without authentication and run or save simulations after registering an account.
- **COMPADRE** — an external scientific database of plant population projection matrices. Data from COMPADRE is not fetched at runtime; it is loaded once at startup by the seed script and stored locally in the PostgreSQL database.

6.2. Building Block View

The building block view decomposes the system into its three tiers and the internal layers of the API.

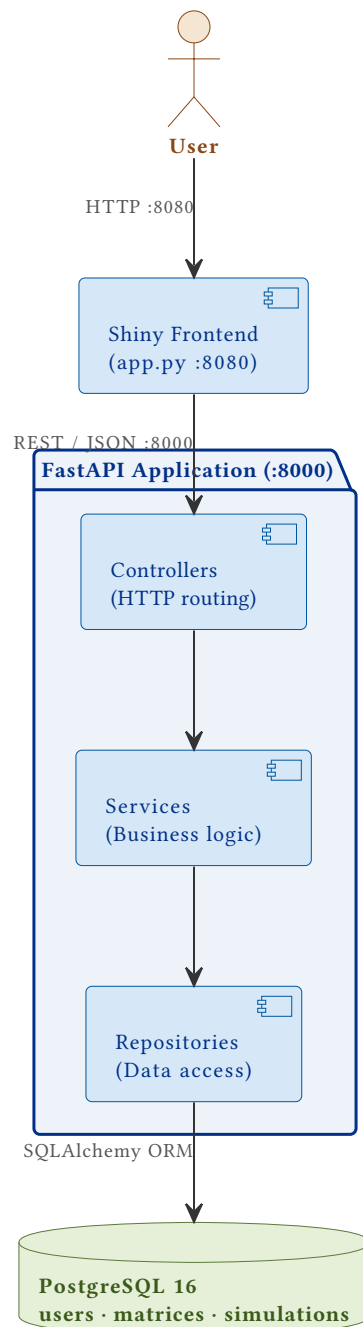


Figure 2: Building block view — tiers and internal API layers

The building block view highlights the strict layer ordering: no layer may communicate with a non-adjacent layer. The frontend is the only component that contacts the API from outside, and the repositories are the only components that touch the database.

6.3. Runtime Views

Each runtime view describes the message flow for one specific use case. Solid arrows represent synchronous calls; dashed arrows represent responses.

6.3.1. User Registration

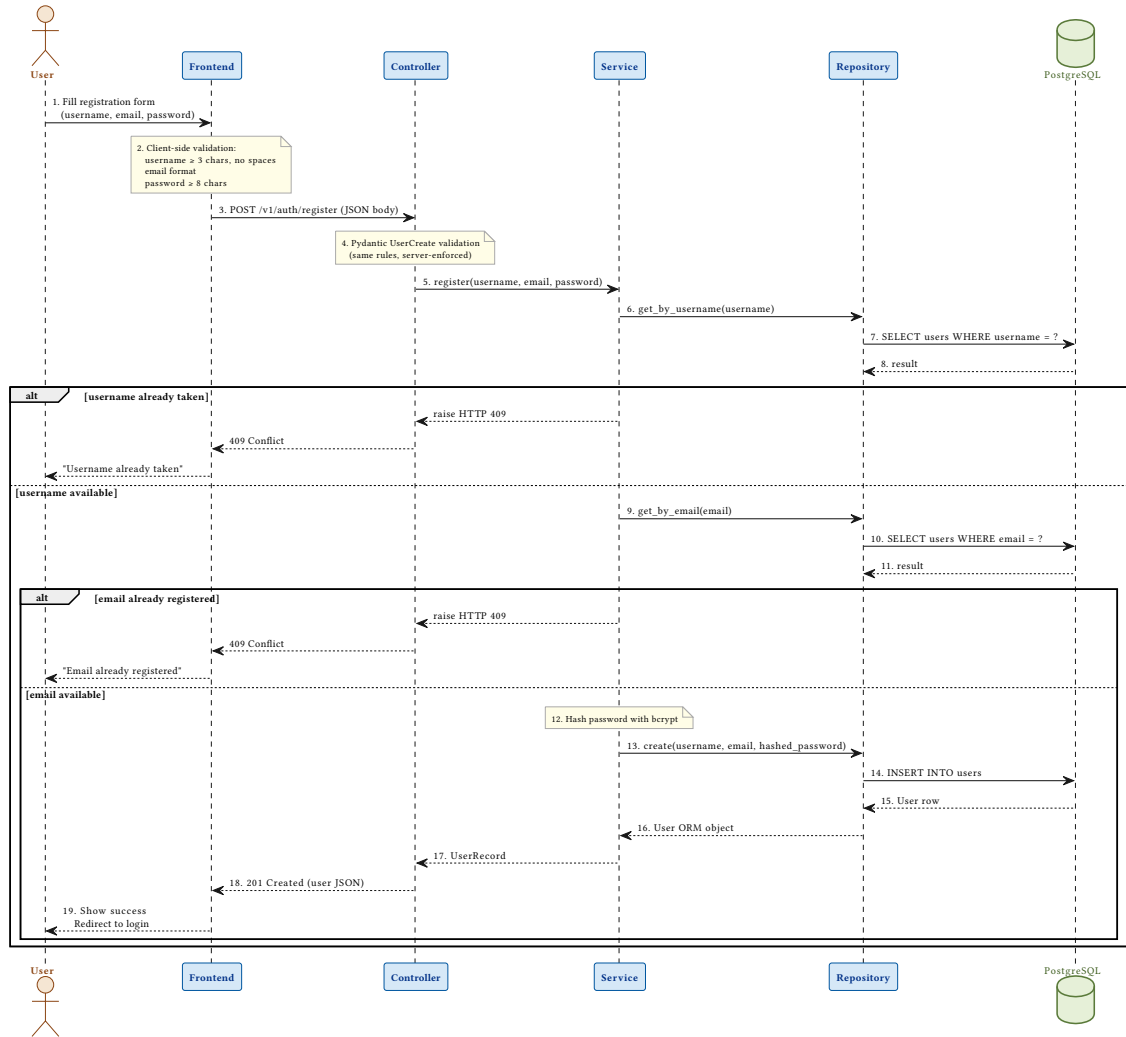


Figure 3: Runtime view — user registration

Registration checks username and email uniqueness before hashing the password with bcrypt and persisting the new account. No JWT is issued at registration; the user must log in separately.

6.3.2. Login

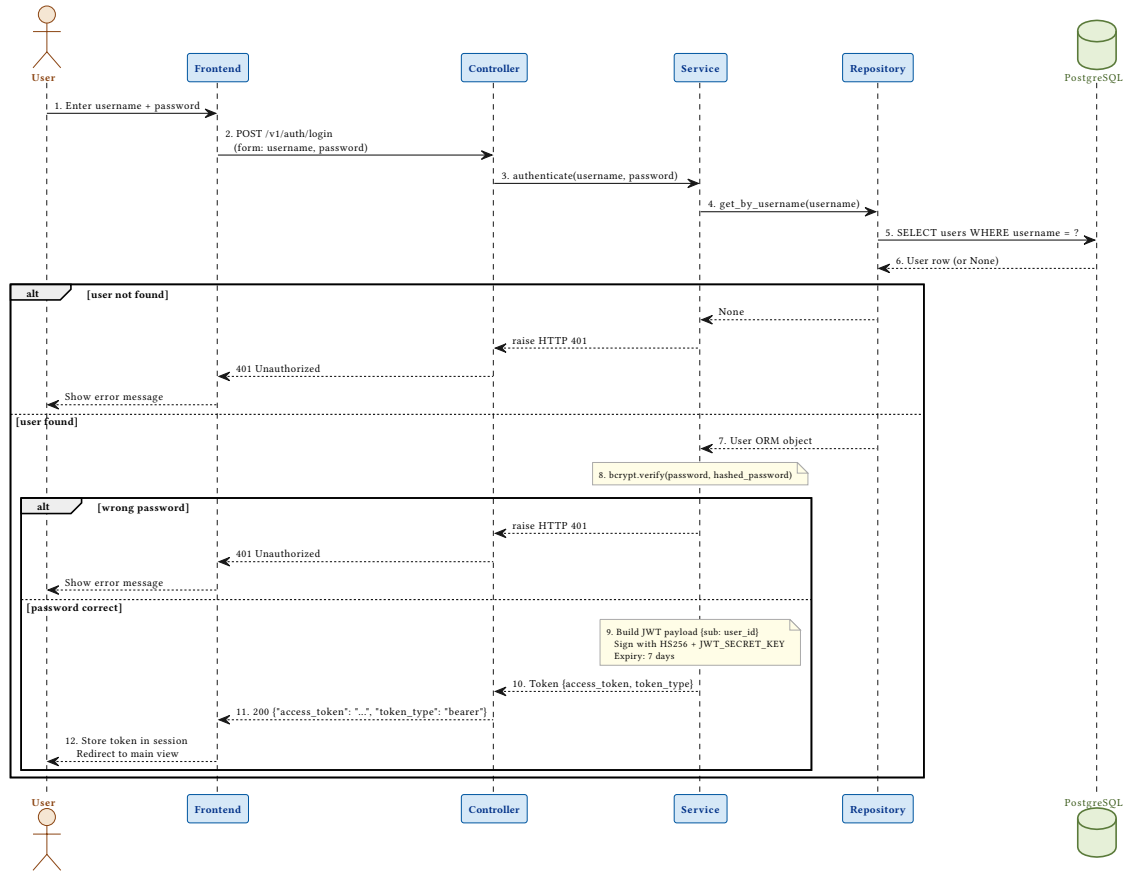


Figure 4: Runtime view — login and JWT issuance

The login endpoint follows the OAuth2 password flow. Credentials are verified by bcrypt comparison against the stored hash. On success a signed JWT with a seven-day expiry is returned and stored client-side by the frontend.

6.3.3. Search and Browse Matrices

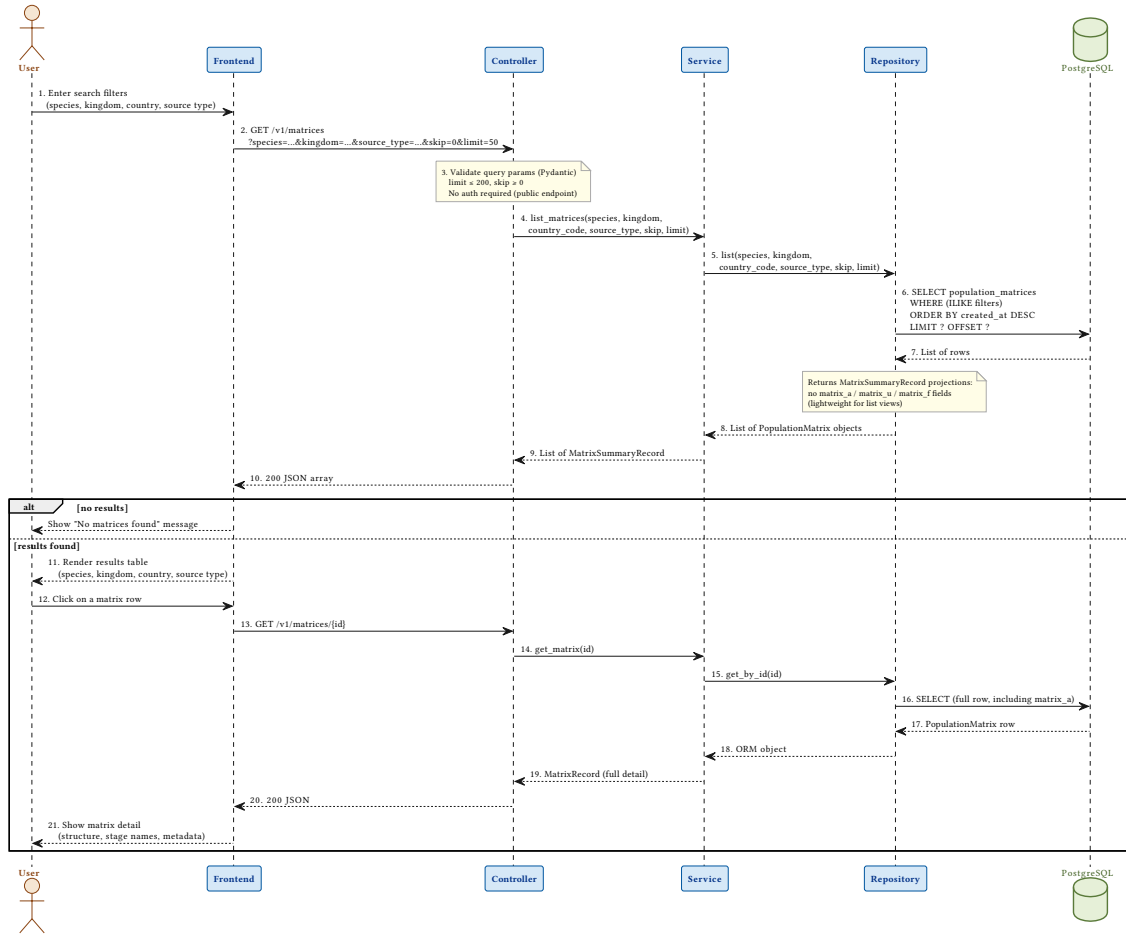


Figure 5: Runtime view — matrix search and detail retrieval

The list endpoint is public — no authentication is required. Filters are applied at the database level using ILIKE for text fields. List responses return `MatrixSummaryRecord` projections (no matrix data) to keep payloads small; the full matrix is fetched only when the user selects a specific entry.

6.3.4. Create a Custom Matrix

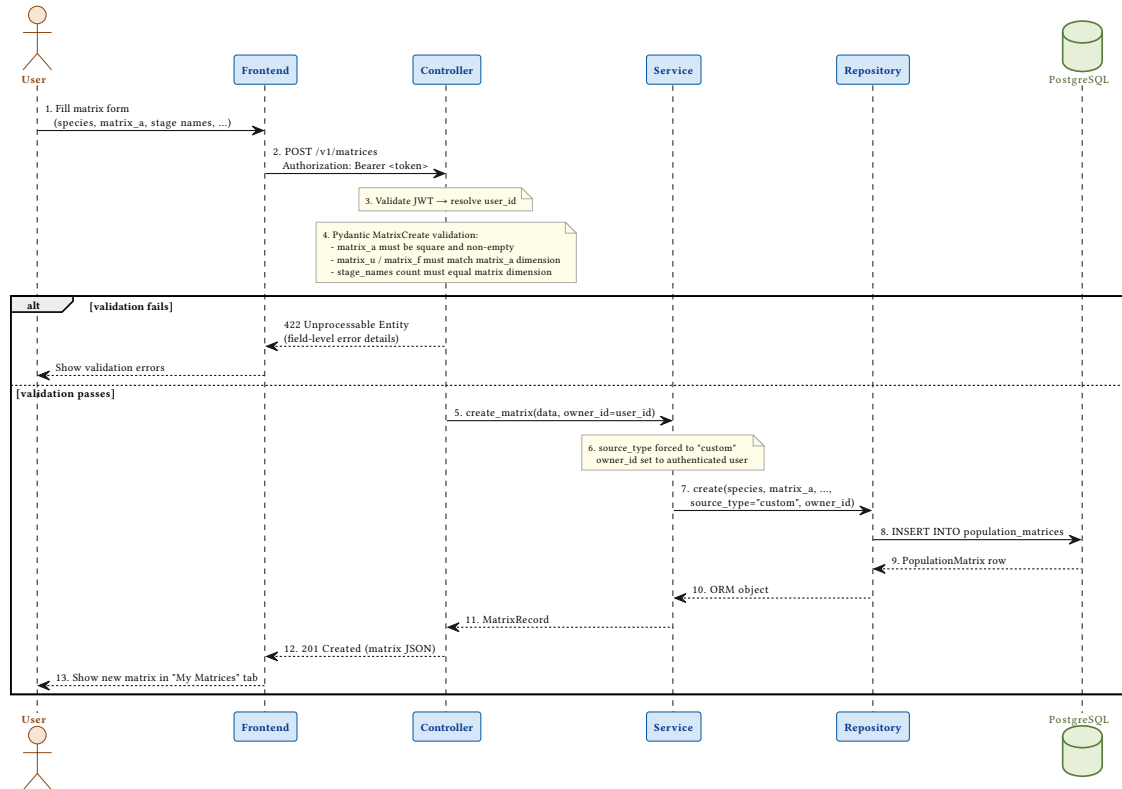


Figure 6: Runtime view — creating a custom matrix

Matrix creation requires authentication. The controller enforces Pydantic shape validation (square matrix, matching sub-matrix dimensions, stage name count) before the service sets `source_type = "custom"` and the requesting user as owner.

6.3.5. Save Simulation

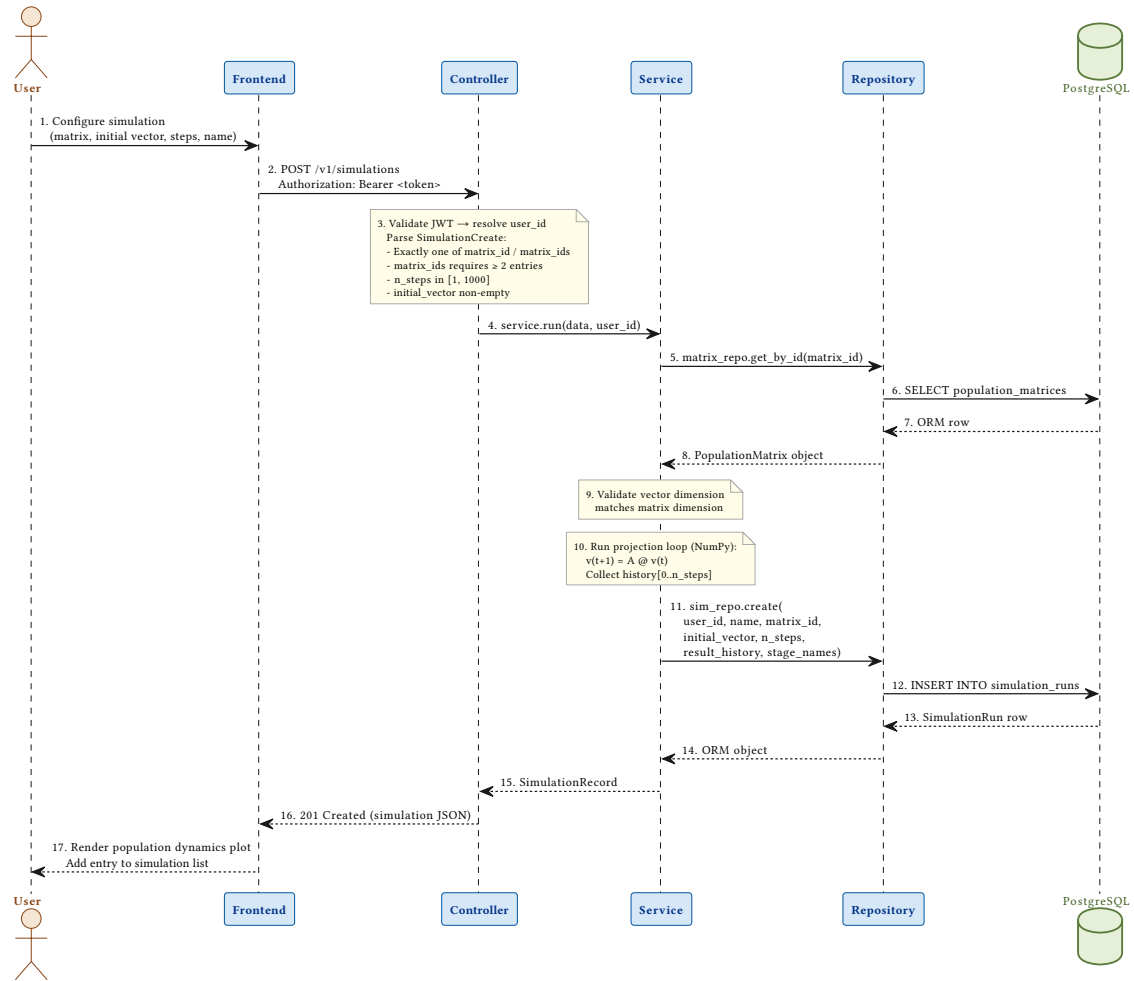


Figure 7: Runtime view — running and storing a simulation

The simulation is computed server-side via a NumPy matrix-vector projection loop and persisted in full. The stored `result_history` contains every population vector from step 0 to step n , making future retrieval self-contained with no need to re-run the algorithm.

6.3.6. Open a Saved Simulation

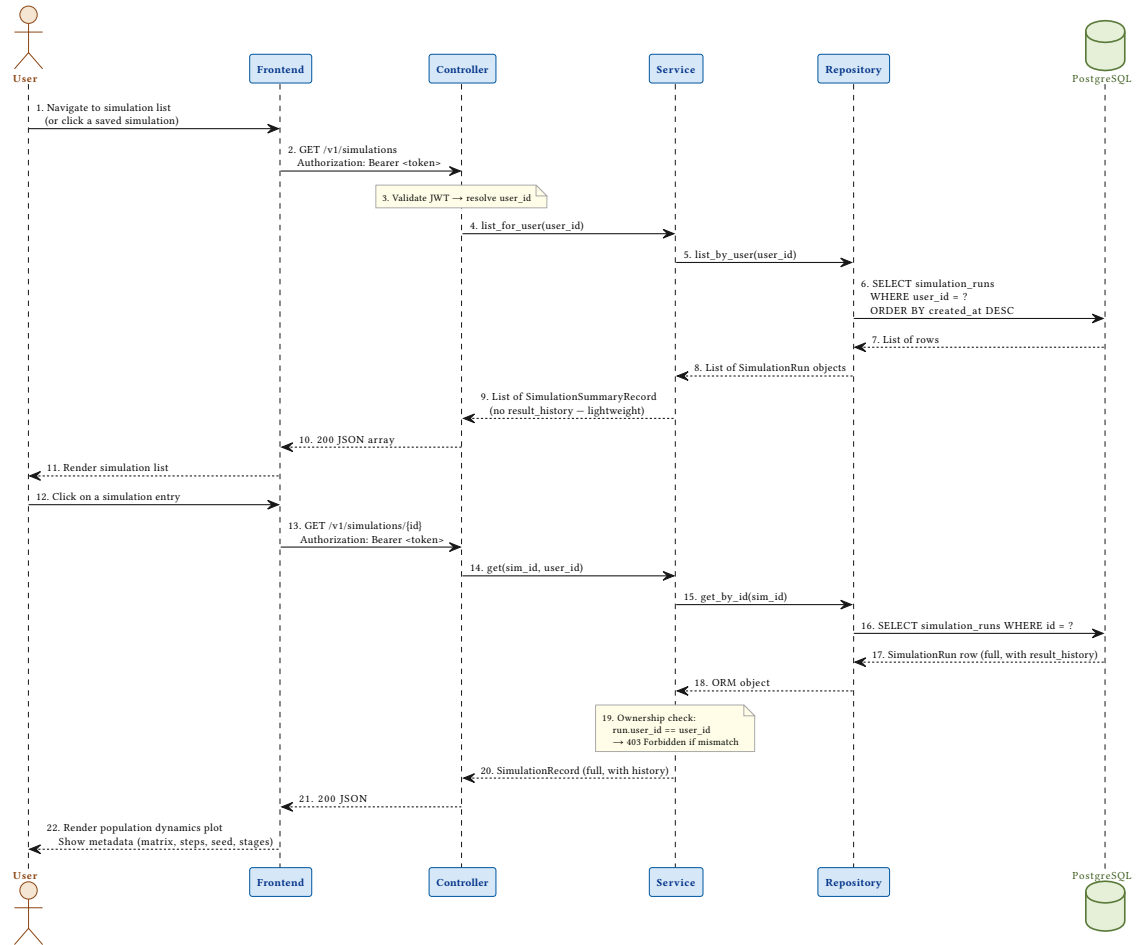


Figure 8: Runtime view — listing and opening a saved simulation

The list endpoint returns lightweight summary records (no `result_history`) for efficiency. The full record, including the complete trajectory, is fetched only when the user selects a specific simulation. An ownership check prevents users from accessing each other's simulations.

6.3.7. Export Simulation

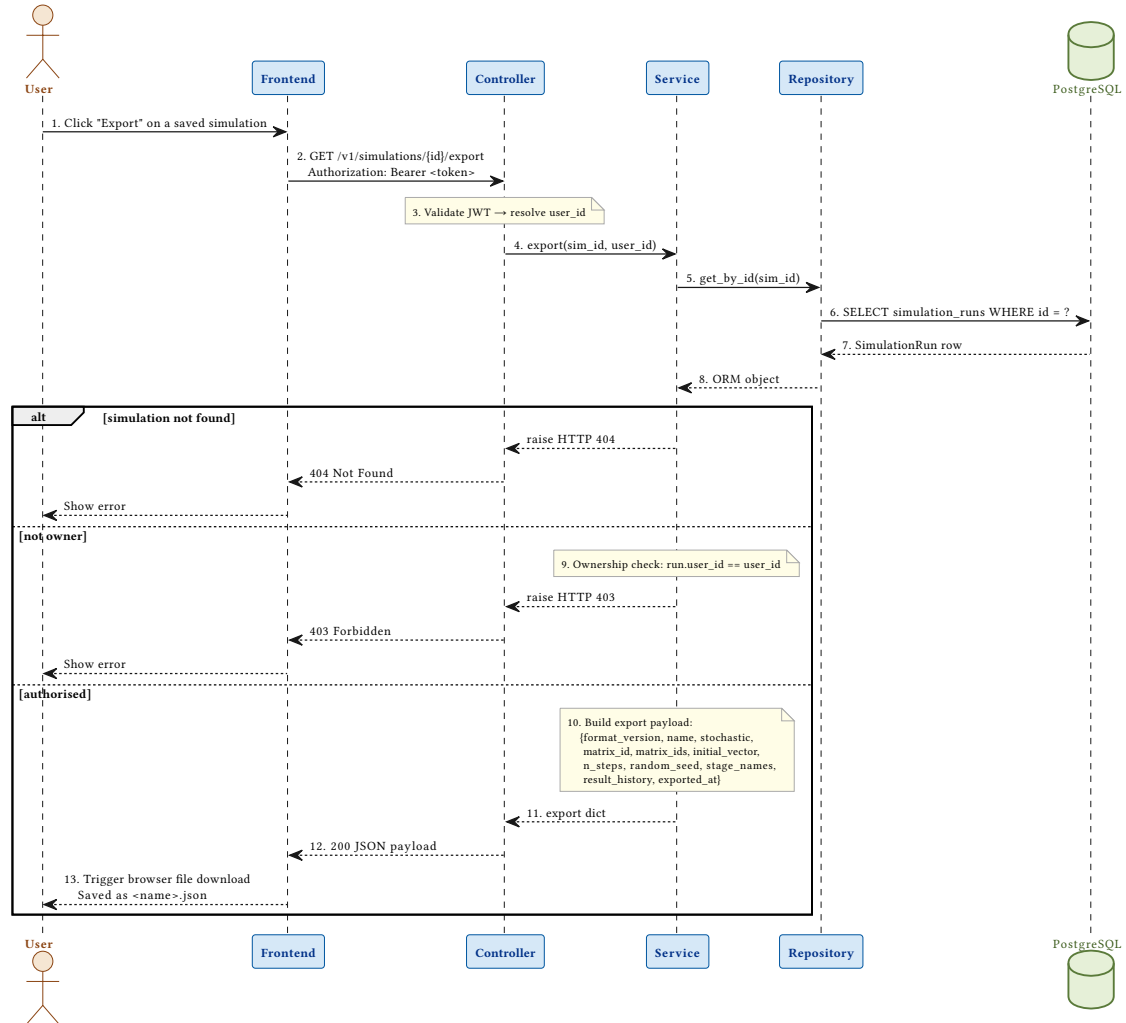


Figure 9: Runtime view — exporting a simulation to a JSON file

The export endpoint builds a self-contained JSON payload from the stored simulation record. The frontend triggers a file download from the response. The exported file includes all information required to re-import the simulation — including the full `result_history` — so no recomputation is needed on import.

6.3.8. Import Simulation

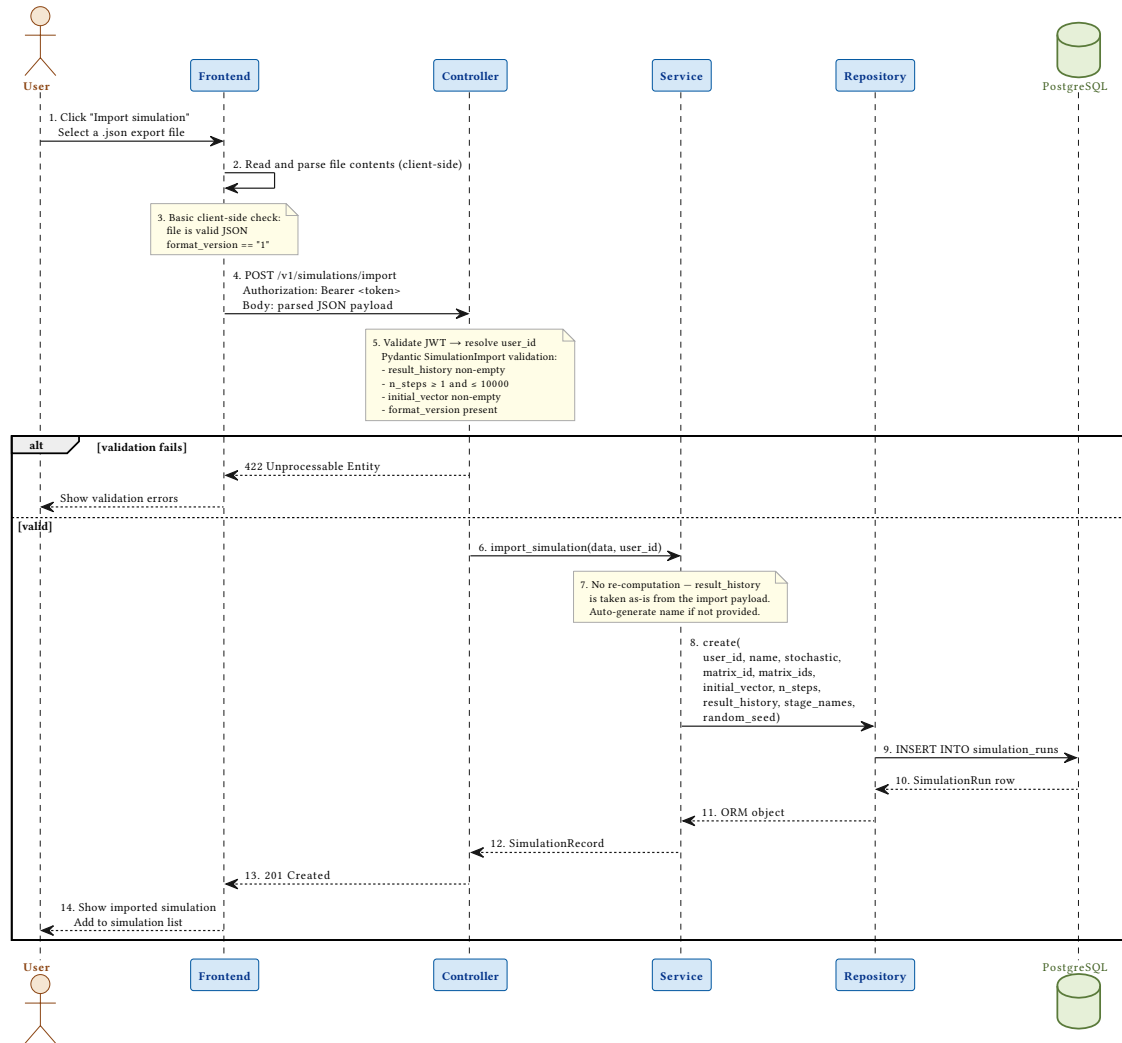


Figure 10: Runtime view — importing a simulation from a JSON file

Import stores the exported payload as a new simulation record without re-running the algorithm. The `result_history` in the file is used as-is. This allows sharing simulation results across accounts or restoring a previously exported run.

7. API Design

7.1. Design Principles

The REST API follows a resource-oriented design. All endpoints are versioned under the `/v1/` prefix so that future breaking changes can be introduced without disrupting existing clients. Resources are nouns (`/matrices`, `/simulations`), and HTTP verbs carry semantic meaning.

The API distinguishes between two categories of endpoints:

- **Public endpoints** — accessible without authentication. These cover read operations on matrices and ephemeral simulation runs, enabling unauthenticated users to explore the dataset and try the simulator.

- **Protected endpoints** — require a valid JWT Bearer token. These cover resource creation, mutation, and operations that are scoped to a specific user (user's simulation library, custom matrix management).

7.2. Endpoint Reference

Method	Path	Auth	Description
GET	/health	No	Liveness check
POST	/v1/auth/register	No	Create a user account
POST	/v1/auth/login	No	Obtain a JWT token
GET	/v1/matrices	No	List / filter matrices
GET	/v1/matrices/{id}	No	Get full matrix detail
POST	/v1/matrices	Yes	Create a custom matrix
PATCH	/v1/matrices/{id}	Yes	Update own matrix
POST	/v1/simulations/run	No	Ephemeral simulation run
POST	/v1/simulations	Yes	Run and store simulation
GET	/v1/simulations	Yes	List own simulations
GET	/v1/simulations/{id}	Yes	Get simulation + history
GET	/v1/simulations/{id}/export	Yes	Export simulation as JSON
POST	/v1/simulations/import	Yes	Import from exported file
DELETE	/v1/simulations/{id}	Yes	Delete own simulation

7.3. Authentication

Authentication is handled with the OAuth2 password flow. The client sends credentials to POST /v1/auth/login as a form-encoded body (compatible with the OAuth2 spec). The server responds with a JWT access token and a token_type of "bearer". Subsequent protected requests must include the token in the Authorization: Bearer <token> header.

Tokens are signed with HS256 using the JWT_SECRET_KEY environment variable and expire after seven days. The get_current_user FastAPI dependency validates the token signature and expiry on every protected request, resolving it to a UserRecord.

7.4. Input Validation — Schemas

All data arriving over HTTP is parsed by Pydantic schema classes defined in api/schemas.py. Validation rules cover:

- **Field-level constraints** — minimum/maximum lengths, numeric bounds (n_steps in [1, 1000]), and non-empty lists.

- **Cross-field invariants** — `SimulationCreate` enforces that exactly one of `matrix_id` (deterministic) or `matrix_ids` (stochastic) is provided, and that `matrix_ids` contains at least two entries when present.
- **Matrix shape validation** — `MatrixCreate` and `MatrixUpdate` verify that `matrix_a` is square and that `matrix_u` / `matrix_f`, when provided, share the same dimension as `matrix_a`.

Validation errors are returned automatically by FastAPI as 422 `Unprocessable Entity` responses with a structured JSON body describing each failing field.

7.5. Output Serialisation — Records

Service methods return Pydantic record objects (defined in `api/records.py`) which FastAPI serialises to JSON. Two projection levels exist for each resource:

- **Summary records** — lightweight projections used in list responses. For matrices these omit `matrix_a`, `matrix_u`, and `matrix_f`. For simulations these omit `result_history` and `initial_vector`, keeping list responses compact.
- **Full records** — complete representations returned by single-resource GET endpoints.

7.6. Ephemeral vs. Stored Simulations

The simulation API exposes two distinct modes.

`POST /v1/simulations/run` performs an ephemeral run: the computation is executed and the full result is returned in the HTTP response, but nothing is persisted. This endpoint requires no authentication and is intended for quick exploration without building up a simulation history.

`POST /v1/simulations` performs a stored run: the computation is executed, the result is written to `simulation_runs`, and the full record is returned. This endpoint requires authentication. The stored simulation can later be retrieved, exported, or deleted.

The export/import pair (`GET /{id}/export` + `POST /import`) allows users to download a simulation as a JSON file and restore it — either to the same account or to a different one — without re-running the computation.

7.7. Ownership and Immutability Rules

Ownership rules are enforced entirely in the service layer, never in the controller or repository.

- A user can only retrieve, export, or delete simulations they own.
- A user can only update matrices they created (`source_type` = "custom").
- COMPADRE matrices (`source_type` = "compadre") are immutable for all users, including authenticated ones. Any PATCH request against a COMPADRE matrix returns 403 `Forbidden` with a descriptive error message.

7.8. Error Responses

The API uses standard HTTP status codes throughout.

Code	Meaning
400	Bad request — business rule violation (e.g. vector dimension mismatch)

Code	Meaning
401	Missing or invalid JWT token
403	Authenticated but not authorised (wrong owner, COMPADRE immutability)
404	Requested resource does not exist
422	Input validation failure — Pydantic schema rejected the request body

8. Database Design

8.1. Schema Overview

The database contains three tables managed by SQLAlchemy ORM models defined in `db/models.py`. Figure 11 shows the entities, their attributes, and the foreign-key relationships between them.

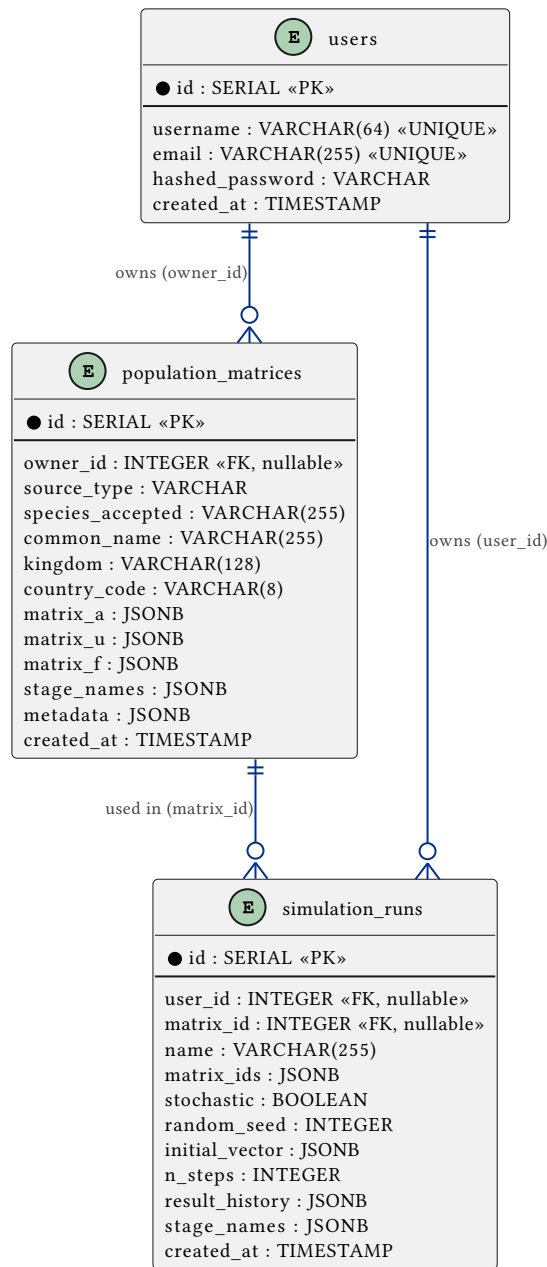


Figure 11: Entity-relationship diagram

8.2. Entity Details

8.2.1. users

Stores registered user accounts. Passwords are stored as bcrypt hashes; the plaintext password is never persisted. The username and email columns carry unique constraints enforced at the database level as a second line of defence after the service-layer check.

8.2.2. population_matrices

The central reference entity. Two kinds of records coexist in this table, distinguished by source_type.

COMPADRE matrices (`source_type = "compadre"`) are seeded from the COMPADRE Plant Matrix Database at startup. They have `owner_id = NULL` and are read-only; any attempt to modify them through the API is rejected with 403

Forbidden. The `metadata_` column stores COMPADRE-specific fields (e.g. `MatrixID`, `StudyDuration`) as a JSONB object.

Custom matrices (`source_type = "custom"`) are created by authenticated users through `POST /v1/matrices`. They are owned by the creating user (`owner_id` is set to the user's id) and can be updated or used in simulations.

The ORM column is named `metadata_` (with a trailing underscore) to avoid a name collision with SQLAlchemy's internal `metadata` attribute. The API serialises it as `"metadata"` through a Pydantic `validation_alias`.

8.2.3. simulation_runs

Records a complete simulation result including every vector in the trajectory. The `result_history` JSONB column stores the full list of population vectors from step 0 (the initial vector) to step `n_steps`, inclusive. This makes retrieval of individual runs self-contained without re-running the algorithm.

For deterministic runs, `matrix_id` is set and `matrix_ids` is null. For stochastic runs, `matrix_ids` is set (as a JSONB array of integers) and `matrix_id` is null. The `random_seed` column enables exact reproduction of stochastic runs.

The `stage_names` column is a snapshot of the stage labels at run time. This ensures that historical simulations remain interpretable even if the source matrix is later updated.

8.3. Migrations

Schema evolution is managed by Alembic. Each migration is a versioned Python script in `alembic/versions/` containing `upgrade()` and `downgrade()` functions. The current migration chain is:

```
0001_initial_schema.py - users, population_matrices
0002_simulation_runs.py - simulation_runs with stochastic columns
```

Migrations are applied automatically by `entrypoint.sh` before the API server starts:

```
python -m alembic upgrade head
```

Running `alembic upgrade head` is idempotent; already-applied migrations are skipped. This guarantees that the schema is always up to date when the container starts, without any manual intervention in any environment.

To generate a new migration after modifying `db/models.py`:

```
python -m alembic revision --autogenerate -m "description"
```

Alembic compares the current ORM model definitions against the live schema and generates the necessary `ALTER TABLE` statements automatically.

8.4. COMPADRE Seeding

`db/seed_compadre.py` populates the `population_matrices` table with real population projection matrices from the COMPADRE database. The seeder is called by `entrypoint.sh` immedi-

ately after migrations complete. It is idempotent — it checks for existing COMPADRE records before inserting — so restarting the container does not duplicate data.

COMPADRE matrices provide users with a ready-made set of ecologically validated matrices to explore and use in simulations without needing to create their own.

9. Simulation Engine

9.1. Mathematical Background

The simulator is built on the theory of **stage-structured population models**, also known as matrix population models [12]. In this framework a population is partitioned into discrete *stages* (e.g. seedling, juvenile, adult) and its state at any time t is represented as a column vector $\mathbf{v}(t) \in \mathbb{R}^n$, where n is the number of stages and each entry $v_{i(t)}$ is the abundance of individuals in stage i .

The dynamics are governed by a **population projection matrix** $\mathbf{A} \in \mathbb{R}^{n \times n}$, where entry a_{ij} encodes the expected contribution of an individual in stage j to stage i over one time step (one year in most COMPADRE matrices). The projection rule is:

$$\mathbf{v}(t+1) = \mathbf{A}\mathbf{v}(t)$$

Iterating this rule for T steps produces the **trajectory**:

$$\{\mathbf{v}(0), \mathbf{v}(1), \dots, \mathbf{v}(T)\}$$

This trajectory is what the system stores as `result_history`.

9.2. Deterministic Simulation

A deterministic simulation uses a single fixed matrix $\mathbf{A}$ for every time step. Given an initial vector $\mathbf{v}(0)$ and a number of steps T , the algorithm is:

```
A = np.array(matrix_a, dtype=float) # None cells → 0.0
v = np.array(initial_vector, dtype=float)
history = [v.tolist()]
for _ in range(n_steps):
    v = A @ v
    history.append(v.tolist())
```

The resulting history list has length $T + 1$ (one entry per time point, including $t = 0$).

The matrix $\mathbf{A}$ may contain `null` cells, which arise in COMPADRE data when certain vital rates were not measured. These are converted to `0.0` before the computation, treating unmeasured transitions as absent.

9.3. Stochastic Simulation

A stochastic simulation models **environmental stochasticity** by providing a set of $k \geq 2$ matrices $\{\mathbf{A}_1, \mathbf{A}_2, \dots, \mathbf{A}_k\}$, each representing the population dynamics under a different environmental scenario (e.g. a good year vs. a bad year). At each time step one matrix is selected uniformly at random and applied:

$$\mathbf{v}(t+1) = \mathbf{A}_{i_t} \mathbf{v}(t), \quad i_t \sim \text{Uniform}(\{1, \dots, k\})$$

The algorithm is:

```
rng = np.random.default_rng(random_seed)
arrays = [np.array(m, dtype=float) for m in matrices]
v = np.array(initial_vector, dtype=float)
history = [v.tolist()]
for _ in range(n_steps):
    A = arrays[rng.integers(len(arrays))]
    v = A @ v
    history.append(v.tolist())
```

Using `numpy.random.default_rng` (the modern NumPy Generator API) instead of the legacy `numpy.random` module provides statistically independent, reproducible streams. When `random_seed` is provided, the same seed always produces the same trajectory, enabling exact reproduction of past runs. When `random_seed` is `None`, a random seed is drawn from the OS entropy source.

All matrices in a stochastic run must share the same dimension n ; this is validated by the service layer before the algorithm runs.

9.4. Validation

The service layer performs the following checks before invoking either algorithm.

1. **Matrix existence** — every referenced matrix id must exist in the database. Missing matrices return 404 Not Found.
2. **Matrix data presence** — `matrix_a` must not be null. A matrix with no projection data cannot be simulated. Returns 400 Bad Request.
3. **Dimension consistency (stochastic only)** — all matrices in `matrix_ids` must have the same n . Mismatched dimensions return 400 Bad Request with a message listing the conflicting dimensions.
4. **Vector dimension match** — `len(initial_vector)` must equal n . A mismatch returns 400 Bad Request with the expected and actual sizes.

9.5. Ephemeral vs. Stored Runs

`SimulationService` exposes two entry points for running a simulation.

`run_ephemeral` computes the trajectory and returns it directly without writing anything to the database. It is called by `POST /v1/simulations/run`.

`run` computes the trajectory and then calls `SimulationRepository.create` to persist the full result. It is called by `POST /v1/simulations`.

The algorithm is identical in both cases — only the persistence step differs.

9.6. Key Properties

Determinism. Given the same matrix, initial vector, number of steps (and seed for stochastic runs), the algorithm always produces the same trajectory. This is guaranteed by NumPy’s BLAS routines for the deterministic path and by `default_rng` for the stochastic path.

Performance. Each projection step is a single BLAS Level-2 matrix-vector product. For the matrix sizes present in the COMPADRE dataset (typically $n \leq 20$) this is computed in microsec-

onds. Even with $T = 1000$ steps the computation is negligible compared to network and I/O latency.

None handling. `_to_array` converts every `None` cell to `0.0` before constructing the NumPy array. This makes the engine robust to incomplete COMPADRE entries without requiring callers to pre-process the matrix data.

10. Testing Strategy

10.1. Philosophy

The test suite is designed around two guiding principles.

Fast feedback. Unit tests that require no external dependencies run first in CI and can also be run locally in seconds. This allows developers to validate logic changes without needing a running database.

Realistic coverage. Integration tests exercise the full HTTP stack against a real PostgreSQL instance, catching issues that unit tests with mocked repositories cannot detect (SQL query correctness, migration state, serialisation round-trips).

10.2. Test Layout

```
tests/
  conftest.py           - shared fixtures (DB setup, TestClient, users)
  test_auth.py          - integration tests for /v1/auth/*
  test_matrices.py      - integration tests for /v1/matrices/*
  test_simulations.py   - integration tests for /v1/simulations/*
  test_health.py        - integration test for /health
  unit/
    test_schemas.py    - unit tests for Pydantic schema validation
    test_matrix_service.py - unit tests for MatrixService (mocked repo)
    test_auth_service.py - unit tests for AuthService (mocked repo)
    test_simulation_service.py - unit tests for SimulationService (mocked repos)
```

10.3. Unit Tests

Unit tests live in `tests/unit/` and run with no database. Repositories are replaced with `unittest.mock.MagicMock` instances whose return values are configured per test case. This isolates the service layer's logic from persistence concerns.

10.3.1. Schema Validation Tests (`test_schemas.py`)

These tests exercise the Pydantic schema classes directly by constructing them with various valid and invalid payloads and asserting the resulting `ValidationError` messages or the parsed model fields.

10.3.2. Service Tests (`test_matrix_service.py`, `test_auth_service.py`, `test_simulation_service.py`)

Service tests follow a common pattern:

1. Create a `MagicMock` for each repository the service depends on.
2. Configure the mock's return values to represent the desired database state.
3. Instantiate the service with the mock repositories.

4. Call the service method and assert on the return value or on the exception raised.
5. Assert that the mock's methods were (or were not) called with the expected arguments.

The simulation service unit tests additionally exercise the core static methods directly:

- `_to_array` — verifies that `None` cells map to `0.0` and that the array dtype is `float`.
- `_compute_deterministic` — verifies history length, initial vector preservation, identity matrix behaviour, and a manual single-step product.
- `_compute_stochastic` — verifies history length, seed reproducibility, and that different seeds produce different trajectories.
- `_validate_vector` — verifies that a correct dimension passes silently and that a mismatch raises HTTP 400.

10.4. Integration Tests

Integration tests use FastAPI's `TestClient` backed by a real PostgreSQL database (`matrix_db_test`). The test database is created fresh at the start of the test session and dropped at the end. All tables are truncated before each individual test, guaranteeing complete isolation between tests.

10.4.1. Shared Fixtures (`conftest.py`)

`conftest.py` provides the following fixtures, available to all integration tests.

Fixture	Description
<code>client</code>	Truncates tables, wires the test DB, yields a <code>TestClient</code>
<code>alice</code>	Registered user + auth headers
<code>bob</code>	A second registered user for ownership tests
<code>alice_matrix</code>	A custom matrix owned by alice (via API)
<code>compadre_matrix_id</code>	A COMPADRE matrix inserted directly into the test DB

10.4.2. Simulation Integration Tests (`test_simulations.py`)

The simulation integration tests cover:

- **Ephemeral runs** — public access, correct history length, first element equals initial vector, reproducibility with the same seed, all error cases (matrix not found, vector mismatch, dimension mismatch between stochastic matrices, schema validation failures).
- **Stored runs** — authentication requirement, deterministic and stochastic persistence, auto-naming, invalid token rejection.
- **List** — authentication requirement, ownership isolation (each user sees only their own simulations), empty list, absence of `result_history` in list items.
- **Get** — full history returned, authentication requirement, 404 and 403 cases.
- **Export / import** — correct JSON structure, round-trip (export then re-import produces a second stored record), missing field rejection.
- **Delete** — resource removal confirmed by a subsequent GET, ownership guard, authentication requirement.

10.5. Running the Tests

```
# Unit tests only – no database needed
python -m pytest tests/unit/ -v

# Full suite – requires PostgreSQL from .env
python -m pytest tests/ -v

# Single file
python -m pytest tests/test_simulations.py -v

# By name pattern
python -m pytest tests/ -k "stochastic"

# With coverage
python -m pytest tests/ --cov=api --cov=db --cov-report=term-missing
```

10.6. Coverage

Coverage is measured with `pytest-cov` and reported to Codecov on every push to main. The coverage run includes both unit and integration tests to produce a combined report covering the `api/` and `db/` modules. The CI pipeline generates both a terminal summary and an XML report consumed by the Codecov action.

11. DevOps and CI/CD

11.1. Overview

Two GitHub Actions workflow files drive the automated pipeline.

File	Trigger	Purpose
ci.yml	Push / PR to main	Correctness — tests and coverage
security.yml	Push / PR to main, weekly schedule	Security — SAST, dependency audit, container scan

Separating correctness from security into distinct workflow files achieves two goals. First, it prevents a security job failure from blocking a merge when the fix requires a dependency update that is out of the developer's immediate control. Second, the security workflow runs on a weekly schedule independently of code changes, surfacing newly disclosed CVEs in dependencies or base images that no code commit could have detected.

11.2. CI Workflow (ci.yml)

The CI workflow runs on every push and pull request targeting main. It provisions a PostgreSQL 16 service container so that integration tests can run in the same job without any external infrastructure.

11.2.1. Steps

1. **Checkout** — `actions/checkout@v4`.
2. **Set up Python 3.13** — `actions/setup-python@v5` with pip caching.

3. **Install dependencies** — `pip install -r requirements.txt pytest pytest-cov`.
4. **Run database migrations** — `python -m alembic upgrade head`, applying all pending migrations to the CI PostgreSQL instance.
5. **Run unit tests** — `pytest tests/unit/ -v --tb=short`. These complete in seconds and provide fast feedback before slower integration tests run.
6. **Run integration tests** — `pytest tests/ --ignore=tests/unit/ -v --tb=short`.
7. **Generate coverage report** — a combined run with `--cov=api --cov=db` produces both a terminal summary and an XML report.
8. **Upload to Codecov** — the XML report is sent to Codecov for trend tracking and badge generation. `fail_ci_if_error: false` prevents a Codecov outage from blocking the pipeline.

11.2.2. Environment Variables

The CI job sets the following environment variables to configure the application against the provisioned PostgreSQL container.

```
POSTGRES_USER: postgres
POSTGRES_PASSWORD: postgres
POSTGRES_DB: matrix_db
POSTGRES_HOST: localhost
POSTGRES_PORT: 5432
JWT_SECRET_KEY: ci-test-secret-key-not-used-in-production
```

11.3. Security Workflow (security.yml)

The security workflow runs on push, pull request, and every Monday at 08:00 UTC. It is composed of three independent jobs.

11.3.1. Bandit SAST

Bandit [13] performs static analysis of the Python source in `api/` and `db/` looking for common security anti-patterns (e.g. hardcoded passwords, use of insecure hash functions, SQL injection risks).

The job runs in two passes.

Audit pass — scans at medium severity/confidence and writes the findings to a JSON artifact (`bandit-report.json`) uploaded as a workflow artifact. This run uses `|| true` to avoid failing the job, ensuring the artifact is always produced.

Gate pass — re-scans at high severity/confidence without `|| true`. If Bandit finds any high-severity, high-confidence issue, the job fails and the merge is blocked.

11.3.2. pip-audit

`pip-audit` [14] checks `requirements.txt` against the PyPA Advisory Database for known CVEs in any direct or transitive dependency. The report is written as JSON and uploaded as a workflow artifact. The job fails if any vulnerability is found, prompting an immediate dependency update.

11.3.3. Trivy Container Scan

Trivy [15] scans the built Docker image for OS-level and library-level vulnerabilities at CRITICAL and HIGH severity. The image is built from the project `Dockerfile` before scanning, so the scan reflects the exact image that would be deployed.

The job is configured with `exit-code: "0"` so that findings are reported to the job log without failing the pipeline. This is a deliberate choice: OS vulnerabilities in the base image may not have fixes available immediately, and blocking every push on an unpatched upstream CVE would be counterproductive. The weekly schedule ensures that the report is reviewed regularly.

11.4. Docker Setup

The project ships three Docker-related files.

`Dockerfile` — builds the API image. It installs Python dependencies from `requirements.txt` and copies the application source. The `entrypoint.sh` script is set as the container entrypoint.

`frontend/Dockerfile` — builds the Shiny frontend image.

`docker-compose.yml` — defines the full three-container stack and wires services together. Notable details.

- The `db` service uses a named volume to persist data across container restarts.
- The `api` and `frontend` services declare `depends_on` with a health check condition, so they only start after PostgreSQL is ready to accept connections.
- The `POSTGRES_PORT` environment variable is overridden to 5432 inside the containers (the value in `.env` is 5435, the host-side port), preventing a misconfiguration that would cause the application to connect to the wrong port when running inside Docker.

11.4.1. `entrypoint.sh`

The API container entrypoint performs three steps before starting the server.

1. Apply pending Alembic migrations: `python -m alembic upgrade head`.
2. Seed COMPADRE data: `python db/seed_compadre.py`.
3. Start Uvicorn: `python -m uvicorn api.main:app --host 0.0.0.0 --port 8000`.

Steps 1 and 2 are idempotent. Restarting the container does not duplicate data or re-apply already-applied migrations.

Important: `entrypoint.sh` must have Unix line endings (LF). Editing this file on Windows with a tool that writes CRLF line endings will cause the script to fail with a cryptic interpreter error. The convention is to run `sed -i 's/\r//'` `entrypoint.sh` after any Windows-side edit.

11.5. Local Development

The full stack can be started locally with:

```
make up          # build images, apply migrations, seed COMPADRE, start services
make logs        # follow all logs
make down        # stop containers (data preserved)
make clean       # stop containers and delete all volumes
```

On Windows where `make` is not available the underlying Docker Compose commands can be used directly:

```
docker compose up -d --build
docker compose logs -f api
docker compose down
```

Services after startup:

Service	URL
Frontend	http://localhost:8080
API	http://localhost:8000
Swagger UI	http://localhost:8000/docs

12. Cross-Cutting Concepts

Cross-cutting concepts are architectural concerns that affect multiple components and cannot be assigned to a single layer or module. This chapter documents them following arc42 section 8.

12.1. Security

12.1.1. Authentication and Authorisation

Authentication is implemented with JSON Web Tokens (JWT) signed with the HS256 algorithm. The `JWT_SECRET_KEY` environment variable holds the signing secret and is never hardcoded or committed to the repository. Tokens carry the user identity (sub claim set to the user id) and expire after seven days.

Every protected endpoint declares the `get_current_user` FastAPI dependency. This dependency validates the token signature, checks expiry, and resolves the authenticated user before any controller logic runs. If the token is missing, malformed, or expired the dependency raises HTTP 401 immediately, and the request never reaches the service layer.

Authorisation is handled exclusively in the service layer. Ownership checks (`run.user_id == user_id`, `matrix.owner_id == user_id`) are explicit Python comparisons, not SQL WHERE clauses. This keeps the rules visible, auditable, and fully testable with mocked repositories without a database.

12.1.2. Password Storage

User passwords are hashed with `bcrypt` before being written to the database. The plaintext password exists only for the duration of the registration or login request and is never logged or persisted. The `hashed_password` column in the `users` table stores only the `bcrypt` digest.

12.1.3. Input Validation as a Security Boundary

All data entering the system over HTTP is parsed and validated by Pydantic schema classes before reaching any business logic. This provides a strict security boundary at the API surface:

- Unexpected fields are ignored.
- Type coercion is performed explicitly (no implicit casting from string to int).
- Cross-field invariants (e.g. exactly one of `matrix_id` / `matrix_ids`) are checked before any database access occurs.

Validation failures are returned as 422 Unprocessable Entity with structured field-level error messages, preventing the service layer from ever receiving malformed input.

12.1.4. Static and Dependency Security Analysis

Three automated security tools run on every push and pull request and on a weekly schedule:

- **Bandit** performs Python SAST, scanning `api/` and `db/` for common security anti-patterns. A two-pass strategy is used: a medium-severity pass produces an artifact for review, and a high-severity pass gates the pipeline.
- **pip-audit** scans `requirements.txt` against the PyPA Advisory Database for known CVEs in direct and transitive dependencies.
- **Trivy** scans the built Docker image for OS-level and library-level vulnerabilities at CRITICAL and HIGH severity.

12.2. Error Handling

12.2.1. Uniform Exception Strategy

Business rule violations are raised as `fastapi.HTTPException` inside the service layer with an explicit HTTP status code and a human-readable detail message. Controllers never catch and re-wrap these exceptions — FastAPI propagates them directly to the client as structured JSON error responses.

The following status codes are used consistently throughout the API:

Code	Condition
400	Business rule violation — e.g. vector dimension mismatch, missing <code>matrix_a</code>
401	Missing, malformed, or expired JWT token
403	Authenticated but not authorised — wrong owner, COMPADRE immutability
404	Requested resource does not exist
409	Uniqueness conflict — username or email already registered
422	Pydantic schema validation failure — structured field-level detail

12.2.2. No Silent Failures

Every error path in the service layer raises an exception explicitly. There are no bare `except` blocks that swallow errors, no `None` returns where a resource is expected to exist, and no fallback values that mask unexpected states. This makes the system's behaviour predictable and the error surface easy to test.

12.3. Observability

12.3.1. Request Logging

Uvicorn and FastAPI emit structured access logs by default for every HTTP request, capturing the method, path, response status code, and processing time. These logs are written to standard output and collected by Docker.

12.3.2. Health Endpoint

The `GET /health` endpoint returns a static 200 OK response with no authentication required. It serves as a liveness probe for Docker Compose health checks, load balancers, and monitoring tools. The API container is only considered ready after this endpoint responds successfully.

12.3.3. Test Coverage Reporting

The CI pipeline measures test coverage with `pytest-cov` on every push to main and uploads the XML report to Codecov. This provides a continuous, visible measure of how much of `api/` and `db/` is exercised by the test suite, and surfaces coverage regressions in pull request reviews.

12.4. Reproducibility

12.4.1. Stochastic Simulation Seeds

Stochastic simulations store the `random_seed` value used to initialise the NumPy random generator alongside the computed trajectory. Any stored stochastic run can be reproduced exactly by re-submitting the same `matrix_ids`, `initial_vector`, `n_steps`, and `random_seed` to `POST /v1/simulations/run`. This is also verified by the unit test suite, which asserts that identical seeds always produce identical histories.

12.4.2. Versioned Database Migrations

All schema changes are captured as Alembic migration scripts in `alembic/versions/`. Migrations are applied automatically by the container entrypoint on startup, making the database schema deterministic with respect to the deployed code version. Running the same migration chain on any clean PostgreSQL instance always produces the same schema.

12.4.3. Containerised Environment

The full stack is defined declaratively in `docker-compose.yml`. Any machine with Docker installed can reproduce the exact same runtime environment — same Python version, same OS packages, same PostgreSQL version — with a single command. This eliminates environment-specific behaviour and ensures that the CI environment matches the development environment.

12.5. Data Integrity

12.5.1. COMPADRE Immutability

Matrices seeded from the COMPADRE database carry `source_type = "compadre"`. The service layer rejects any PATCH request against a COMPADRE matrix with HTTP 403, regardless of the requesting user's identity. This rule is enforced in `MatrixService.update_matrix` and is covered by dedicated unit and integration tests. No database constraint is relied upon for this invariant — the service layer is the single point of enforcement.

12.5.2. Stage Names Snapshot

When a simulation is stored, the `stage_names` of the source matrix are copied into the `simulation_runs.stage_names` column. This snapshot decouples the historical simulation record from the source matrix: if the matrix is later updated or deleted, the simulation remains fully interpretable with the correct stage labels.

12.5.3. Ownership Isolation

Each user's simulation library is private. The list, get, export, and delete endpoints all perform an ownership check in the service layer before returning or modifying any data. A user who guesses another user's simulation id receives HTTP 403, not the simulation data.

12.6. Testability

12.6.1. Layer Isolation by Design

The controller / service / repository separation is not only an architectural preference — it is the mechanism that makes the business logic unit-testable without a database. The service layer receives its repositories through constructor injection, which allows tests to substitute MagicMock objects with configured return values. This pattern is used throughout `tests/unit/test_simulation_service.py`, `tests/unit/test_matrix_service.py`, and `tests/unit/test_auth_service.py`.

12.6.2. Two-Speed Test Suite

Unit tests in `tests/unit/` run in seconds with no external dependencies and provide fast feedback during development. Integration tests in `tests/` exercise the full HTTP stack against a real PostgreSQL instance and validate SQL correctness, migration state, and serialisation round-trips. The CI pipeline runs unit tests first, only proceeding to integration tests if the unit tests pass.

12.7. Configuration Management

12.7.1. Environment Variables

All environment-specific values are externalised to a `.env` file that is never committed to the repository. The required variables are:

Variable	Purpose
POSTGRES_USER	Database username
POSTGRES_PASSWORD	Database password
POSTGRES_DB	Database name
POSTGRES_HOST	Database host (localhost outside Docker, db inside)
POSTGRES_PORT	Host-side port (5435); overridden to 5432 inside Docker
JWT_SECRET_KEY	HMAC secret for JWT signing — must be kept confidential

12.7.2. Docker Port Override

The `.env` file sets `POSTGRES_PORT=5435`, which is the host-side port exposed by the db container. Inside Docker, however, PostgreSQL listens on the standard port 5432. The `docker-compose.yml` overrides `POSTGRES_PORT` to 5432 for the `api` and `frontend` services so that they connect correctly without requiring a separate environment file for Docker. This is an explicit design decision documented in `CLAUDE.md` to prevent the misconfiguration from being reintroduced.

Bibliography

- [1] S. Ramírez, “FastAPI.” 2018.
- [2] S. Colvin and contributors, “Pydantic — Data validation using Python type hints.” 2017.

- [3] C. R. Harris, K. J. Millman, S. J. van der Walt, and others, “Array programming with NumPy,” *Nature*, vol. 585, pp. 357–362, 2020, doi: 10.1038/s41586-020-2649-2.
- [4] M. Bayer and contributors, “SQLAlchemy — The Python SQL Toolkit and Object Relational Mapper.” 2006.
- [5] M. Bayer and contributors, “Alembic — Database migrations for SQLAlchemy.” 2011.
- [6] Posit PBC, “Shiny for Python.” 2022.
- [7] Docker Inc., “Docker — Accelerated container application development.” 2013.
- [8] Docker Inc., “Docker Compose.” 2014.
- [9] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT).” [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc7519>
- [10] H. Krekel and contributors, “pytest — helps you write better programs.” 2004.
- [11] GitHub Inc., “GitHub Actions — Automate your workflow.” 2019.
- [12] H. Caswell, *Matrix Population Models: Construction, Analysis, and Interpretation*, 2nd ed. Sunderland, MA: Sinauer Associates, 2001.
- [13] PyCQA, “Bandit — A security linter for Python.” 2014.
- [14] PyPA, “pip-audit — Audit Python environments for known vulnerabilities.” 2021.
- [15] Aqua Security, “Trivy — A comprehensive and versatile security scanner.” 2019.